

1. Super Store

Description:

Build an online shopping interface where buyers and sellers can engage in efficient transactions across a wide range of products.

Modules:

- **Profile Service**
- **Inventory Service**
- **Order Service**
- **Payment Service**

Requirements:

- Allow buyers and sellers to register and log in.
 - Allow sellers to manage their inventory of products.
 - Operations: `addItem()`, `updateItem()`
 - Allow buyers to add items to the cart, place orders, and make payments.
 - Operations: `listInventory()`, `buyItem()`, `addToCart()`, `makePayment()`
 - Validate orders against inventory.
 - Password validation and encryption.
-

2. Call Taxi Booking Application

Description:

Design a taxi booking application where customers can book taxis available at certain points in a linear route.

Problem Assumptions:

- **Taxi Count:** Assume 4 taxis for simplicity, but it should scale to any number of taxis.
- **Points on Route:** A, B, C, D, E, F (each 15 km apart)
- **Travel Time Between Points:** 60 minutes
- **Charges:**
 - Minimum Rs.100 for the first 5 km
 - Rs.10 per km thereafter
- **Initial Taxi Position:** All taxis are stationed at A.

Booking Rules:

- When a customer books a taxi:
 - A free taxi at the pickup point is allocated.
 - If no free taxi is available, the nearest taxi is allocated.
 - If two taxis are free at the same point, the one with lower earnings is allocated.
 - Taxis only charge from the pickup point to the drop point.
 - If no taxi is available, the booking is rejected.

Modules:

1. Call Taxi Booking

Sample Inputs and Outputs:

Input 1:

- Customer ID: 1
- Pickup Point: A
- Drop Point: B
- Pickup Time: 9

Output 1:

- Taxi can be allotted.
- Taxi-1 is allotted.

Input 2:

- Customer ID: 2
- Pickup Point: B
- Drop Point: D
- Pickup Time: 9

Output 2:

- Taxi can be allotted.
- Taxi-2 is allotted.

Input 3:

- Customer ID: 3
- Pickup Point: B
- Drop Point: C
- Pickup Time: 12

Output 3:

- Taxi can be allotted.
- Taxi-1 is allotted.

2. Display Taxi Details

Output Example:

Taxi-1		Total Earnings: Rs. 400			
BookingID	CustomerID	From	To	PickupTime	DropTime
1	1	A	B	9	10
3	3	B	C	12	13
Taxi-2		Total Earnings: Rs. 350			
BookingID	CustomerID	From	To	PickupTime	DropTime
2	2	B	D	9	11

3. Railway Reservation System

Description:

Develop a railway reservation application with the following types of seats:

- **AC Coach**
- **Non-AC Coach**
- **Seater**

Each seat type has a capacity of 60 seats with a waiting list max of 10 seats.

Modules:

1. **Booking**
 2. **Availability Checking**
 3. **Cancellation**
 4. **Prepare Chart**
-

4. Matrix Game with Ray Rules

Description:

A matrix-based game where rays interact with atoms inside a grid according to specific rules.

Problem Setup:

- **Grid:**

R3		-	-	-	
R2		-	-	-	
R1		-	-	-	
		C1	C2	C3	

- **Rules:**
 - **Rule 1:** A ray with an atom in its path prints 'H' (Hit).
 - **Rule 2 & 3:** A ray with an atom in a diagonal adjacent position refracts.
 - **Rule 4:** A ray with atoms in both diagonal positions reflects.
 - **Rule 5:** Deflection happens in the order of input rays.

Sample Input and Output:

Input:

- Position of atoms and the rays originating from outside the box.
- Example:

```
3
3 1
2 2
1 3
3
R3 C1 C3
```

Output:

- Display the final state of the box.

Sample Matrix Display:

	C1	H	C3	
R3	-	-	-	R3
H	-	X	-	H
R1	-	-	-	R1
	C1	H	C3	

Here's a refined version of the question based on your description:

5. Brick Breaker

1. Array Structure:

You have a 7x7 grid array representing a game where:

- `w` represents walls surrounding the area.
- `g` represents ground.
- `o` represents the ball.
- Numbers (1, 2, 3, etc.) represent bricks with each number indicating the strength of the brick (i.e., how many hits it can take before disappearing).

2. Initial Setup:

- **Input size:** 7
- **Number of bricks:** 6
- **Positions of bricks:** (2, 2), (2, 3), (2, 4), (3, 2), (3, 3), (3, 4)

3. Game Mechanics:

- The ball starts with 5 lives.
- The ball can move in three directions:
 - **St (Straight):** The ball moves vertically upwards.
 - **Lt (Left Diagonal):** The ball moves diagonally left-upward.
 - **Rt (Right Diagonal):** The ball moves diagonally right-upward.
- When the ball hits a brick, the brick's strength decreases by 1. If a brick's strength reaches 0, it disappears.
- After hitting a brick, the ball returns to its original position unless the brick disappears, in which case it moves into the empty space left by the brick.

4. Commands:

- **St (Straight):** If there is a brick in the ball's path, it hits it, decreasing the brick's strength. The ball then returns to its starting position if the brick remains. If the brick disappears (reaches 0 strength), the ball moves into the position where the brick was.

- **Lt (Left Diagonal):** The ball moves diagonally left. If it encounters a brick, it hits it, reducing the brick's strength by 1. If the brick disappears, the ball takes its position and continues moving downward.
- **Rt (Right Diagonal):** The ball moves diagonally right, following the same rules as Lt.

5. Example Gameplay:

- **Input Grid:**

```

w w w w w w w
w                               w
w   1 1 1   w
w   1 1 1   w
w                               w
w                               w
w                               w
w g g o g g w

```

- **Example Commands and Output:**

○ **Input:** St

■ **Output:**

```

w w w w w w w
w                               w
w   1 1 1   w
w   1 1 1   w
w                               w
w                               w
w                               w
w g g o g g w
Ball count: 5

```

Explanation: The ball moved straight up but encountered no bricks, so it returned to its initial position.

○ **Input:** 1t

■ **Output:**

```
W W W W W W W
W
W 1 1 1 W
W 1 1 W
W
W
W
W g o g g g W
Ball count: 4
```

Explanation: The ball moved diagonally left. Since there was no brick immediately in the way, it hit the wall and reflected horizontally, encountering a brick at (3, 2). After hitting the brick, the ball moved downward, causing the brick to disappear. The ball count is now reduced by 1.

6. Advanced Modules:

- In later modules of the game:
 - Each brick has a value indicating its strength. Every hit reduces the strength by 1, and the brick disappears only when its strength reaches 0.
 - The game consists of 7 modules, each adding new mechanics or rules to enhance gameplay.

6. Social Media Application

1. Description:

Design and implement a console-based Social Media Application that allows users to sign up, connect with friends, post updates, interact with posts, and manage notifications.

The system should provide features for user management, friend

suggestions, post interactions, and notifications, ensuring a smooth console-based user experience.

2. **Modules:**

- User Management
- Friend Suggestions
- Friend Request Management
- Post Management
- News Feed
- Notification Management

3. **Functional Requirements:**

1. **User Signup:**

- Users can sign up by entering:
 - Name
 - Date of Birth (DOB)
 - Age
 - Location
 - Occupation
- All details are entered via the console.

2. **Friend Suggestions:**

Friend suggestions are displayed in the following order:

- **Mutual Friends:**
 - Display users who share mutual friends.
 - Show the mutual friend count and names (if only one mutual friend).
- **Matching Profiles:**
 - Show users with matching attributes such as:
 - Name
 - Age
 - DOB month
 - Location
 - Sort by the highest match to the lowest match.
- **Other Users:**
 - List remaining users who do not fall in the above two categories.

3. Friend Request Management:

- Users can send friend requests to suggested users or by searching for specific users.
- Display pending friend requests with requester details.
- Allow users to accept or reject friend requests.
- Update friendship status once a request is accepted.

4. View Friends:

- Display the list of friends with their details:
 - Name
 - Age
 - Location
 - Occupation

5. Create Post:

- Users can create posts that are visible to their friends.
- Example Post:
`"Enjoying the sunny day in New York!
#blessed"`

6. News Feed:

- Display posts from the user's friends including:
 - Post content
 - Author
 - Like count
 - Comments
- New posts appear at the top of the feed.

7. Interact with Posts:

- Users can like or comment on posts.
- Each post shows updated like counts and comments in the feed.

8. Share Post:

- Each post includes a share option.
- Shared posts appear in the recipient's notifications.
- Recipients can view, like, comment, and share the post again.

9. Notification Management:

- Notifications are generated for:
 - Friend requests
 - Accepted requests
 - Shared posts
 - Post interactions
- Notifications are cleared when viewed or acted upon.

4. Example Scenarios:

1. User Registration:

User	Name	Age	Location	Occupation
A	Alice	25	New York	Engineer
B	Bob	25	San Francisco	Designer

2. Friend Suggestions:

- User A sees:
 - Users with mutual friends
 - Matching profiles (same age or location)
 - Other users

3. Sending Friend Requests:

- User A sends a friend request to User B.
- User B receives a notification:
`"Alice sent you a friend request."`

4. Accepting Friend Requests:

- User B views and accepts the request.
- The system updates their friendship status.

- Both users now appear in each other's friend lists.

5. Creating Posts:

- User A creates a post:
"Enjoying the sunny day in New York!
#blessed"

6. News Feed and Interactions:

- User B's feed shows Alice's post.
- User B likes and comments: "Looks amazing!"
- User A shares the post with User C.

7. Notifications:

- User C receives a shared post notification.
- User C can like, comment, or share the post again.

8. Sample Solution (up to Friend Request Management):

[View Solution on Google Drive](#)
